



SAVEETHA
SCHOOL OF ENGINEERING
Affiliated to AICTE | IET-UK Accreditation



SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

**SPELL-CHECKING APPLICATION – CHOOSING THE
BEST DATA STRUCTURE
CSA0310-DATA STRUCTURE**

Guided by:
W. Deva priya

Presented by:
Priyadharshini M (192521051)



INTRODUCTION

- A **spell-checking application** checks whether a typed word is present in a dictionary and identifies spelling errors.
- Since the dictionary may contain thousands or millions of words, fast searching is essential.
- A **Trie (Prefix Tree)** is the most suitable data structure because it stores words character by character, allowing quick word lookup and prefix searching.
- It also supports features such as **autocomplete** and **spell correction**, making it an efficient choice for modern spell-checking systems.





PROBLEM STATEMENT

Problem

A spell-checking application needs to verify whether a typed word exists in its dictionary database.

Requirements

- Fast word lookup
- Efficient handling of large dictionaries
- Quick suggestions for incorrect words
- Low search time



SUITABLE DATA STRUCTURE – TRIE (PREFIX TREE)

Definition

A **Trie** is a tree-based data structure used to store and search strings efficiently.

Features

Each node represents one character.

Words are formed from the root to leaf.

Supports prefix-based searching.

Ideal for dictionaries and autocomplete systems.



WORKING OF TRIE

Root

```
  /  \  
 c   d  
 |   |  
 a   o  
 |   |  
 t   g  
     |  
     s
```

Stored Words:

cat
dog
dogs

Search Example:

Search "cat" → Found 

Search "cow" → Not Found 



JUSTIFICATION BASED ON SEARCH SPEED WHY TRIE?

Searches character by character.

No need to compare the entire dictionary.

Lookup time is **O(L)** regardless of the number of words.

Performance remains fast even with millions of words.

Example

Dictionary:

apple

application

apply

apt

Searching "**apply**"

Traverse only:

$a \rightarrow p \rightarrow p \rightarrow l \rightarrow y$

Search completes in **5 steps**, independent of dictionary size.



WHY TRIE IS THE BEST CHOICE?

- Very fast search
- Search depends only on word length
- Supports autocomplete
- Easy spell correction
- Efficient prefix matching
- Widely used in search engines and dictionaries

Applications

Trie is used in:

- Spell-checking software
- Autocomplete keyboards
- Search engines
- Dictionary applications
- Text prediction
- Contact search



CONCLUSION

- A **Trie (Prefix Tree)** is the most suitable data structure for a spell-checking application because it provides **fast and efficient word searching**.
- Its search time depends only on the **length of the word**, making it ideal for large dictionaries.
- In addition to quick word verification, it supports **autocomplete**, **prefix search**, and **spell correction**, improving both performance and user experience.
- Therefore, a Trie is the preferred choice for modern spell-checking systems.

THANK YOU